@RODRIGOSILVACTO

Workshop de Cursor

Seu Guia Completo para Codificar com Inteligência Artificial

Do Básico ao Avançado

Baseado na Documentação Oficial — cursor.com/docs

Rodrigo Silva

O Cursor Transforma Linguagem Natural em Código Funcional

O Cursor é um editor de código com IA que permite descrever o que deseja construir em linguagem natural.

> Construído sobre a base do VS Code, mantendo toda a familiaridade, mas com superpoderes de IA nativos.

- Autocompletar (Tab)
- Inline Edit Ctrl+K
- Agent Autônomo Ctrl+I
- Terminal Inteligente

models.json

```
// Modelos de IA Suportados
✓ Claude 4.6 Opus
✓ Claude 4.6 Sonnet
✓ GPT-5.2
✓ GPT-5.3 Codex
✓ Gemini 3 Flash
✓ Gemini 3.1 Pro
✓ Grok Code
```

Cada modelo possui diferentes velocidades e capacidades para tarefas específicas.

Os 7 Conceitos Fundamentais que Você Precisa Dominar

CONCEITO	O QUE FAZ	ATALHO / ACESSO
Tab	Autocompletar código com sugestões multi-linha inteligentes.	`Tab`
Agent	IA autônoma que lê, edita e executa código no seu projeto.	`Ctrl` + `I`
Inline Edit	Edita código selecionado via instruções em linguagem natural.	`Ctrl` + `K`
Chat	Interface de conversa com IA, com suporte a múltiplas abas.	`Painel Lateral`
Rules	Instruções persistentes para moldar o comportamento da IA.	`.cursor/rules/`
Context (@)	Mecanismo para fornecer arquivos, docs e histórico à IA.	`@` no chat

Instalação e Primeiros Passos em 5 Minutos

1

Baixe e Instale

Acesse cursor.com e instale no seu sistema operacional (Windows, macOS ou Linux).

2

Importe Configurações

Ao abrir, importe suas extensões, temas e atalhos do VS Code com um clique.

3

Explore com o Agent

Abra o painel com `Ctrl` + `I` e pergunte: "Como este projeto funciona?"

4

Edite com Inline Edit

Selecione um código, pressione `Ctrl` + `K` e descreva a mudança desejada em português.

5

Codifique com Tab

Comece a digitar e use a tecla `Tab` para aceitar sugestões inteligentes de autocompletar.

O Contexto é o Combustível da IA

Domine os @ Mentions para fornecer informações precisas ao Agent.

@Files & Folders

Referencie arquivos inteiros ou pastas completas para dar visibilidade total ao Agent.

Ex: @src/auth.ts ou arraste arquivos da sidebar

@Code

Referencie trechos específicos de código (funções, classes) para controle granular.

Ex: Selecione o código e clique "Add to Chat"

@Docs

Integre documentações oficiais de bibliotecas ou adicione suas próprias docs customizadas.

Ex: @Docs → "Add new doc" → Cole a URL

@Past Chats

Reutilize o contexto de conversas anteriores para continuar trabalhos em andamento.

Ex: @ → Selecione a conversa na lista

Dica Importante

Se você sabe o arquivo exato, use @ para economizar tempo. Se não souber, o Agent encontrará sozinho usando busca semântica no codebase.

Quatro Modos do Agent para Cada Situação

Agent Mode

`Ctrl + I`

O modo padrão. Explora autonomamente o codebase, edita múltiplos arquivos, executa comandos no terminal e corrige erros. Ideal para tarefas complexas e refatoração.

Ask Mode

`Select "Ask"`

Somente leitura. O Agent busca no codebase e responde perguntas sem fazer nenhuma alteração. Perfeito para entender código antes de modificá-lo ou aprender novas partes do sistema.

Plan Mode

`Shift + Tab`

Planejamento estratégico. Pesquisa o codebase e cria um plano detalhado revisável antes de escrever qualquer código. Essencial para features grandes ou mudanças arquiteturais.

Debug Mode

`Ctrl + .`

Caça-bugs avançada. Gera hipóteses, adiciona logs de instrumentação, analisa a execução em runtime e propõe correções direcionadas para problemas difíceis.

Workflow Prático — Do Plano à Execução

CENÁRIO Adicionar autenticação JWT a uma API Express existente.

ETAPA 1

Planejar

Ative o Plan Mode e descreva o objetivo macro.

```
// Atalho: Shift + Tab — "Adicionar auth JWT..."
```

ETAPA 2

Revisar o Plano

O Agent pesquisa o codebase e propõe um plano passo-a-passo.

```
// Ação: Clique em "Save to workspace" (.cursor/plans/)
```

ETAPA 3

Executar

Aprove o plano; o Agent implementará as mudanças automaticamente.

```
// Resultado: Agent gera código e aplica diffs.
```

ETAPA 4

Revisar (Review)

Peça para o Agent analisar o próprio código em busca de bugs.

```
// Comando: Clique em "Review" → "Find Issues"
```

ETAPA 5

Iterar

Refine com instruções ou interrompa se necessário.

```
// Atalho: Escape — "Ajuste o middleware..."
```

Inline Edit — Edição Cirúrgica com Ctrl+K

1. Selecione o código
2. Pressione `Ctrl` + `K`
3. Descreva a mudança

Refatoração

Transforme estruturas de código complexas instantaneamente.

```
"Converta para async/await"
```

Tratamento de Erros

Adicione robustez ao código existente.

```
"Adicione try/catch específico para erros de rede"
```

Documentação

Gere comentários e documentação técnica.

```
"Adicione JSDoc completo com exemplos"
```

```
function fetchUserData(userId) {
return fetch(`/api/users/${userId}`)
.then(response ⇒ response.json())
.then(data ⇒ {
console.log(data);
return data;
});
}
```

Edit — Generating...

Converta esta função para usar async/await ao invés de callbacks

Tab Completion — Autocompletar Inteligente

Contexto Profundo

O Cursor não apenas completa palavras. Ele analisa todo o seu arquivo, arquivos relacionados e edições recentes para prever sua **intenção**.

Previsão Multi-linha

Sugere implementações completas de funções, classes ou testes de uma só vez.

Como Usar

1. Comece a digitar.
2. Veja o texto "fantasma" cinza.
3. Pressione `Tab` para aceitar tudo.

UserService.ts

```
interface UserService {
    getUser(id: string): Promise<User>;
    createUser(data: CreateUserDTO): Promise<User>;
}
class UserServiceImpl implements UserService {
    async getUser(id: string): Promise<User> {
        return db.users.findUnique({ where: { id } });
    }
    async createUser(data: CreateUserDTO): Promise<User> {
        return db.users.create({ data });
    }
}
```

Tab → Accept

Rules — Ensine o Agent a Trabalhar do Seu Jeito

Instruções persistentes que definem o comportamento e padrões da IA.

Project Rules

`.cursor/rules/*.mdc`

Específicas do projeto. Versionadas no Git. Ideais para padrões de código e arquitetura.

User Rules

`Settings > General > Rules`

Globais para você. Aplicam-se a todos os seus projetos.

Team Rules

`Cursor Dashboard`

Para toda a equipe. Sincronizadas automaticamente (Planos Team/Enterprise).

AGENTS.md

`/AGENTS.md`

Alternativa simples na raiz do projeto para instruções gerais em Markdown.

react-patterns.mdc

```
---
description: "Padrões para componentes React"
globs: "src/components/**/*.tsx"
alwaysApply: false
---
# Padrões React do Projeto
- Use functional components com hooks
- Nomeie componentes com PascalCase
- Use TypeScript strict mode
```

ORDEM DE PRECEDÊNCIA: Team Rules > Project Rules > User Rules

Commands — Crie Workflows Reutilizáveis com /

Projeto

`.cursor/commands/`

Global

`~/.cursor/commands/`

Equipe

`Cursor Dashboard`

`.cursor/commands/code-review.md`

```
Revise o código alterado nesta branch seguindo este checklist:

1. Verifique se há vulnerabilidades
2. Confirme validação de inputs
3. Verifique tratamento de erros
4. Confirme cobertura de testes
5. Verifique padrões do projeto

Reporte com severidade (Crítico/Alto/Médio).
```

NO CHAT (CTRL + L)

`/code-review` focando na autenticação

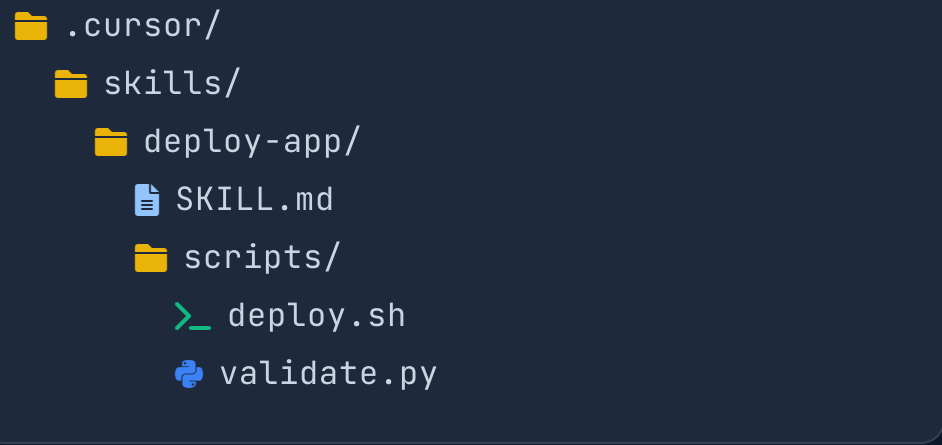
Agent Action

O Agent lê o arquivo de comando, executa o checklist passo-a-passo e gera o relatório solicitado.

Skills — Estenda o Agent com Capacidades Especializadas

Agent Skills é um padrão aberto para empacotar conhecimento de domínio e workflows que o Agent pode usar para tarefas específicas.

ESTRUTURA DE DIRETÓRIOS



SKILL.md

```
---
name: deploy-app
description: Deploy da aplicação para staging ou produção.
---

# Deploy App
Execute o script de deploy:
`scripts/deploy.sh <ambiente>`

Onde <ambiente> é staging ou production.

## Pré-deploy
Antes de deployar, execute:
`python scripts/validate.py`
```

Skills são descobertas **automaticamente** pelo Cursor ou podem ser invocadas com **/** no chat.

Subagents — Orquestre Múltiplos Agentes

Delegue tarefas complexas para agentes especializados que operam em janelas de contexto isoladas, mantendo o foco do Agent principal.

Nativos (Built-in)

Explore

Busca e analisa codebases usando um modelo mais rápido para processar grandes volumes de dados.

Bash

Executa séries de comandos shell, isolando logs verbosos do contexto principal.

Browser

Controla o navegador via MCP, filtrando o ruído de snapshots do DOM.

Customizados

`.cursor/agents/verifier.md`

```
---
name: verifier
description: Valida trabalho completado.
model: inherit
---

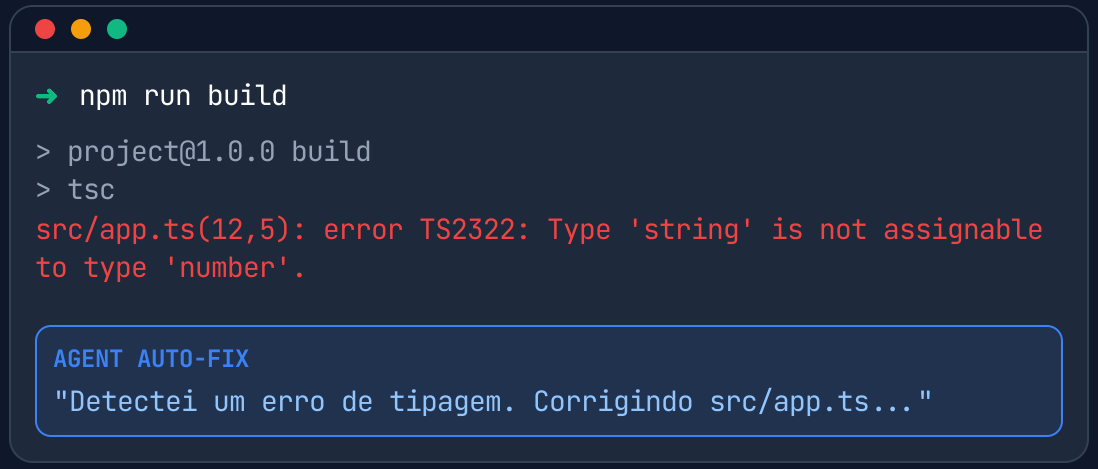
Você é um verificador independente.
Quando invocado:
1. Revise as mudanças feitas
2. Execute os testes relevantes
3. Reporte o que passou vs. incompleto
```

Como Invocar

Use `/verifier` no chat ou peça em linguagem natural: "Use o subagent verifier para validar as mudanças".

Terminal Inteligente e Sandbox — Segurança na Execução

Execução & Auto-Fix



Controle de Segurança

Modo de Aprovação

Exige sua confirmação explícita antes de executar qualquer comando que possa alterar arquivos ou fazer chamadas de rede.

Allowlist de Comandos

Defina comandos seguros que o Agent pode rodar livremente para agilizar o workflow.

`ls` `cat` `grep` `npm test` `git status`

⚠ Comandos destrutivos (rm -rf, git reset --hard) são bloqueados por padrão e exigem aprovação dupla.

Review — Revise Cada Linha Antes de Aceitar

Diff View Claro

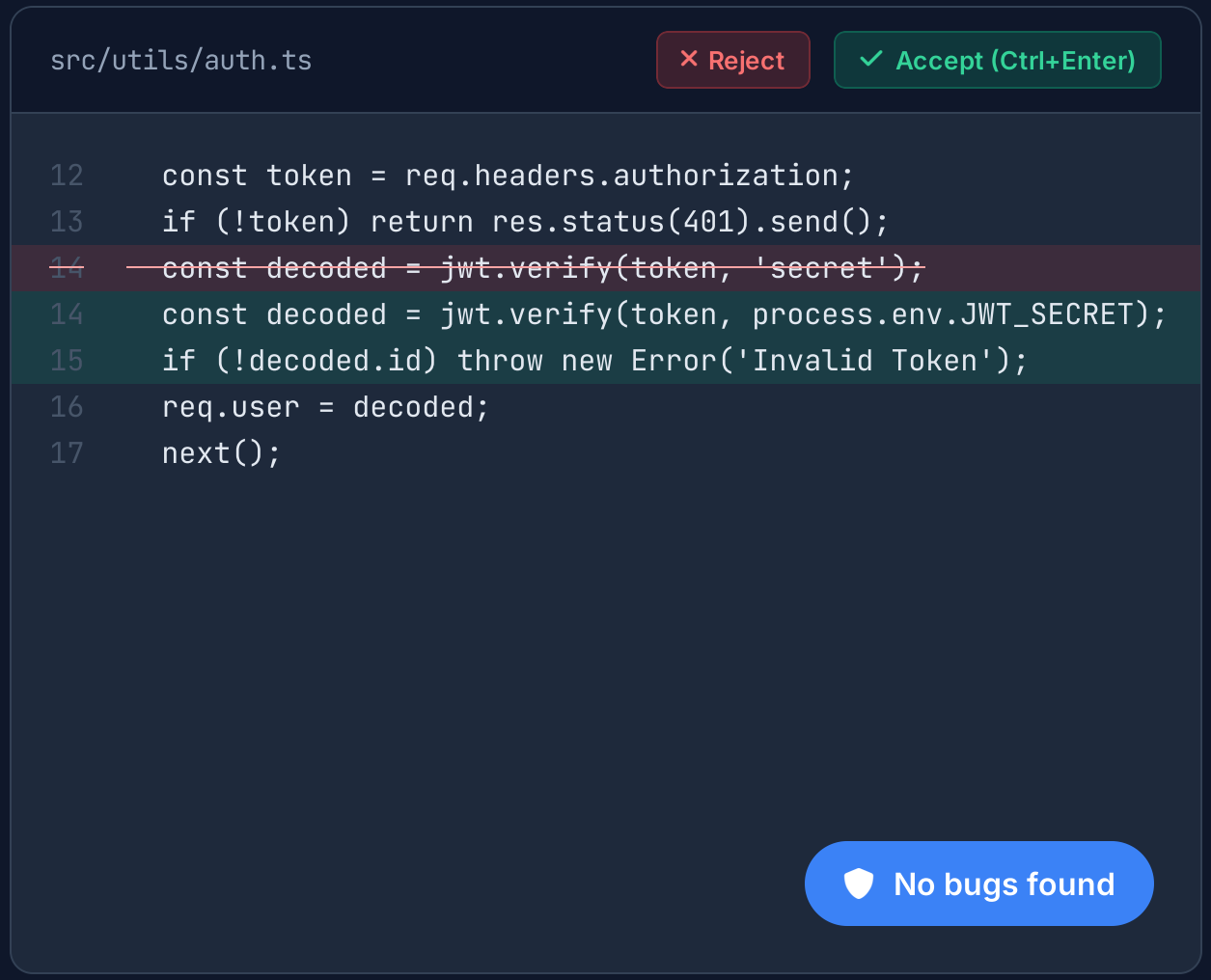
Visualize exatamente o que será removido (vermelho) e adicionado (verde) antes de confirmar.

Aceitação Seletiva

Não é tudo ou nada. Aceite blocos específicos de código e rejeite outros com atalhos rápidos.

Agent Review

Peça para o Agent revisar o próprio código em busca de bugs ou regressões antes de você olhar.



Atalhos Essenciais — Referência Rápida

AI & Agent

`Ctrl` + `K`

Inline Edit. Edite ou gere código no arquivo atual.

`Ctrl` + `L`

Chat. Abra o painel de chat para perguntas e contexto.

`Ctrl` + `I`

Composer. Edição multi-arquivo e criação de features.

`Tab`

Accept. Aceite sugestões de autocompletar ou diffs.

Navegação

`Ctrl` + `P`

Ir para Arquivo (Quick Open).

`Ctrl` + `Shift` + `F`

Busca Global no projeto.

`F12`

Ir para Definição.

`Alt` + `←`

Voltar (Navegação anterior).

Geral

`Ctrl` + `Shift` + `P`

Command Palette (Acesso a tudo).

`Ctrl` + `B`

Alternar barra lateral (Sidebar).

`Ctrl` + `` ` ``

Alternar Terminal integrado.

`Ctrl` + `/`

Comentar/Descomentar linha.

Prompts Eficazes — A Diferença entre Bom e Excelente

FRACO (VAGO)

```
"Crie uma página de login."
```

BOM (ESPECÍFICO)

```
"Crie um componente React de login com email e senha usando Tailwind CSS e validação básica."
```

EXCELENTE (CONTEXTUALIZADO)

```
"@Login.tsx @auth-context.ts Implemente o formulário de login usando react-hook-form e zod. Valide o email corporativo e integre com o método `login` do contexto. Use os componentes de UI existentes em @components/ui."
```

A Fórmula Mágica: Contexto (@Files, @Docs) + Intenção Clara (Verbo de ação) + Restrições (Libs, Padrões)

Exemplo Completo — Configurando um Projeto do Zero

1. Defina o Cérebro do Projeto

`/AGENTS.md`

Crie o arquivo mestre na raiz. Descreva a missão, a stack tecnológica (Next.js, Tailwind, Supabase) e as convenções globais de arquitetura.

2. Estabeleça as Regras

`.cursor/rules/*.mdc`

Crie regras específicas para contextos. Ex: `style-guide.mdc` para CSS e `db-schema.mdc` para SQL.

```
globs: "src/**/*.tsx"
description: "Use Tailwind utility classes, avoid inline styles."
```

3. Automatize Workflows

`.cursor/commands/test.md`

Padronize tarefas repetitivas. Crie um comando para rodar testes e corrigir falhas automaticamente.

4. Validação Inicial

`Chat (Ctrl + I)`

Verifique se o Agent entendeu o contexto. Digite: "@AGENTS.md leia este arquivo e explique a arquitetura do projeto."

Dicas Avançadas para Produtividade Máxima

Contexto Limpo

Chats longos degradam a performance e confundem o modelo. Não tenha medo de começar do zero.

AÇÃO RECOMENDADA

Use `Ctrl + L` para iniciar um novo chat a cada nova tarefa ou feature.

Validação Primeiro

Evite o ciclo de "tentativa e erro". Peça para o Agent criar um teste que falha antes de corrigir o código.

PROMPT EXEMPLO

"Crie um script de reprodução para este bug antes de tentar corrigi-lo."

Seja o Arquiteto

Pare de digitar sintaxe. Foque em descrever a arquitetura, revisar a lógica e orquestrar a solução.

MINDSET

Você é o Diretor Técnico; o Cursor é o Desenvolvedor Sênior.

Recapitulação e Próximos Passos

O Que Dominamos

- Instalação e migração do VS Code em 5 min
- Os 4 Modos: Agent, Ask, Plan e Debug
- Domínio do Contexto com @Mentions
- Personalização via Rules (.mdc) e Skills
- Workflow seguro com Review e Sandbox

Sua Missão Agora

01 **Crie seu AGENTS.md**
Defina a "personalidade" e as regras globais do seu projeto na raiz do repositório.

02 **Configure uma Rule de Projeto**
Crie uma regra em `.cursor/rules/` para padronizar seu estilo de código (ex: React, Python).

03 **Use o Plan Mode**
Na sua próxima feature complexa, use `Shift+Tab` antes de escrever qualquer código.

Rodrigo Silva | Cursor Ambassador | @rodrigosilvacto